LECTURE SERIES · DATABASE SYSTEMS

Indexing & B+ Trees

File organization, ordered indices, multi-level indexing, B+ tree operations & hashing

Md. Rakib Trofder

Lecturer, Bangladesh Army University of Science & Technology

Indexing Basics | Special Indexing | Temporal Indexing | B+ Tree | Insertion / Deletion | Practice

WHY WE INDEX

What Is an Index?

An index is like the index at the back of a book: instead of reading every page, you look up a word and jump straight to its page number.

WITHOUT AN INDEX

1,000

pages read to find one word. The database does the same: a full table scan reads every row.

WITH AN INDEX

2–3

look-ups: find the word in the index, read its page number, open that page.

BOOK INDEX

B+ tree	p. 245
Hashing	p. 262
Index	p. 231
SQL	p. 41

IN A DATABASE, EVERY INDEX ENTRY HAS TWO PARTS

Search key

the column we look up on, e.g. roll

→

Pointer

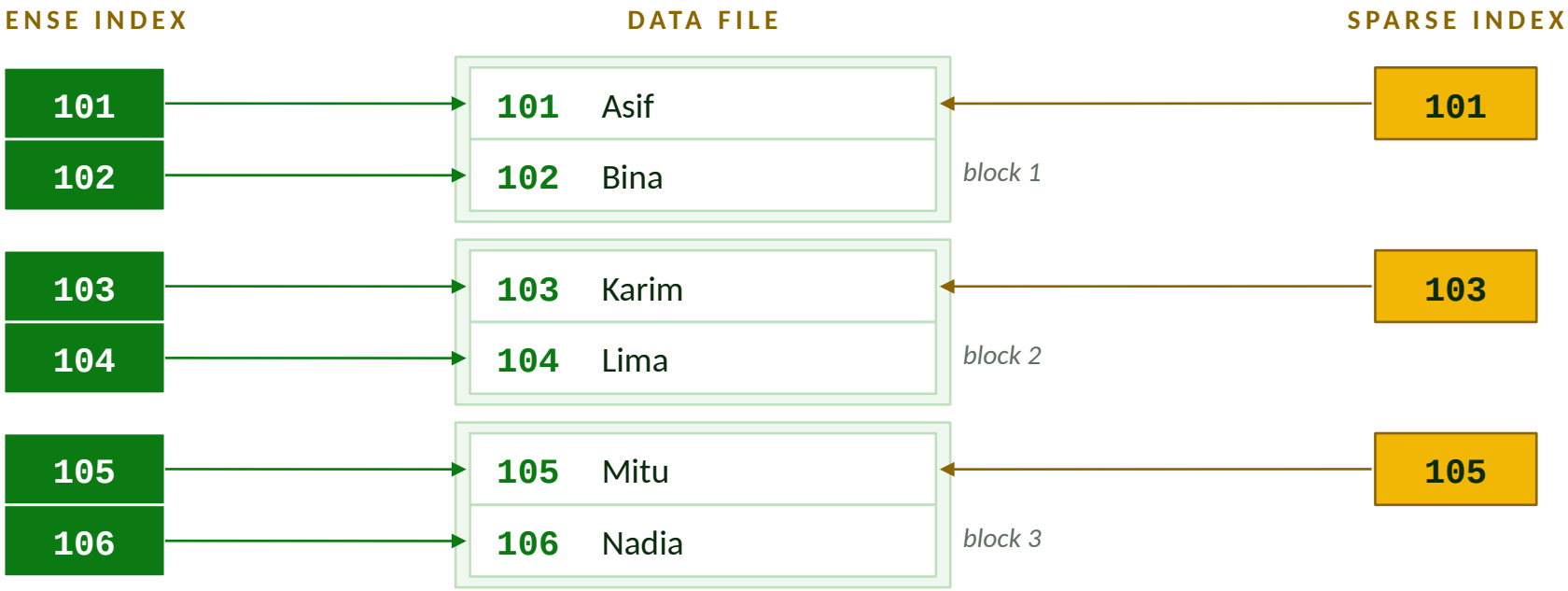
where the row lives (disk block / row address)

```
CREATE INDEX idx_roll ON Student(roll);
SELECT * FROM Student WHERE roll = 2104;
```

TWO WAYS TO BUILD AN INDEX

Dense vs Sparse Index

Example: Student file sorted by roll, stored 2 records per disk block.



Dense: one entry for every key. Direct hit, but a bigger index.

Sparse: one entry per block. Smaller index. To find 104: take the largest entry ≤ 104 (103), go to block 2, scan it.

HOW WE JUDGE ANY INDEX

- Access type
- Access time
- Insertion time
- Deletion time
- Space overhead

INDEX ENTRIES KEPT IN SORTED ORDER

Primary vs Secondary Index

An ordered index stores its entries sorted by the search key. There are two kinds, depending on how the data file itself is sorted.

Primary (Clustering) Index

search key = the order the file is sorted in

- File is physically sorted on this key (e.g. roll)
- A sparse index is enough
- Only one primary index per table
- Great for range queries: rows are next to each other

Example: Index on roll, file sorted by roll

Secondary (Non-clustering) Index

search key ≠ the order the file is sorted in

- File order has nothing to do with this key (e.g. dept)
- Must be dense: every value is indexed
- Many secondary indices allowed per table
- Range queries are slower: matching rows are scattered

Example: Index on dept, file sorted by roll

Key idea: because ordered indices keep keys sorted, they handle both = and BETWEEN queries. The B+ tree is the ordered index used by almost every DBMS.

INDEXING LOCATIONS AND SHAPES

Spatial Indexing

Spatial data has a position in space: points (a shop), lines (a road) or polygons (a district). A spatial index answers **“what is near here?”** quickly.

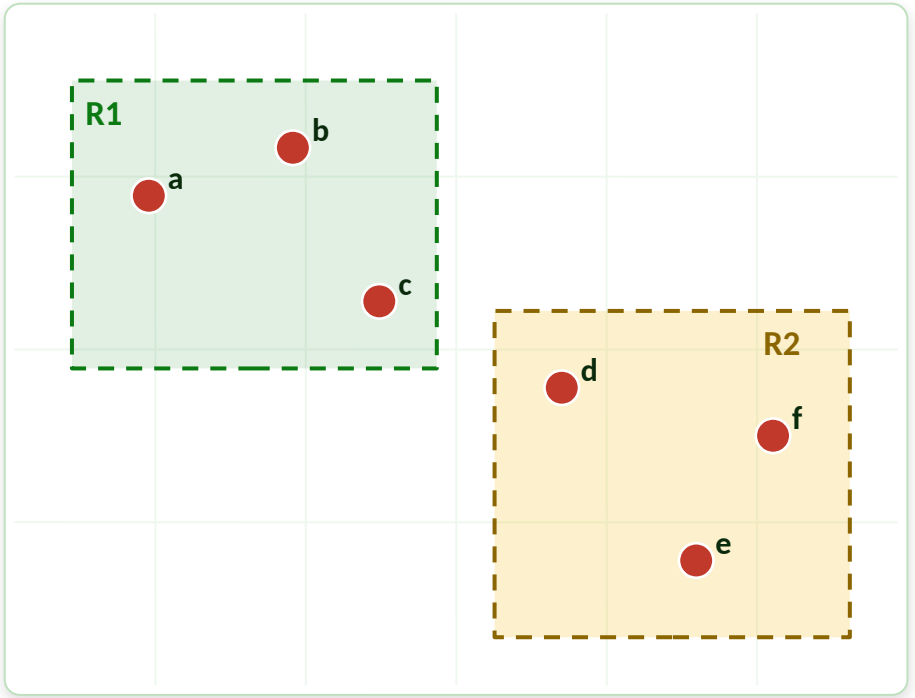
WHY NOT A B+ TREE?

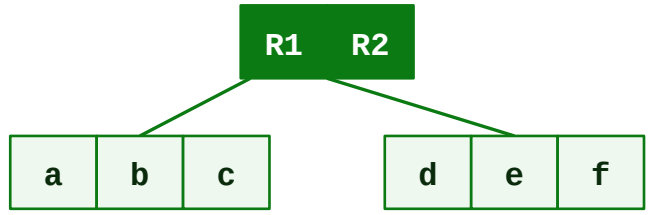
A B+ tree sorts on one value. A location has two (x, y), and there is no single order that keeps nearby places next to each other.

COMMON SPATIAL INDEX STRUCTURES

1. **R-tree —** groups nearby objects inside bounding rectangles; rectangles are nested like a tree (most common)
2. **Quad-tree —** splits the map into 4 squares, then splits busy squares again
3. **Grid / Geohash —** divides the map into fixed cells; each cell gets a code

Used in: Google Maps, Pathao/Uber (“nearest driver”), GIS, PostGIS & MySQL **`SPATIAL INDEX`**





Query “shops near d”: only R2 overlaps, so R1 is skipped.

INDEXING DATA THAT CHANGES OVER TIME

Temporal Indexing

Temporal data keeps history: every row carries a time period during which it was true. A temporal index answers **"what was the value at time T?"** quickly.

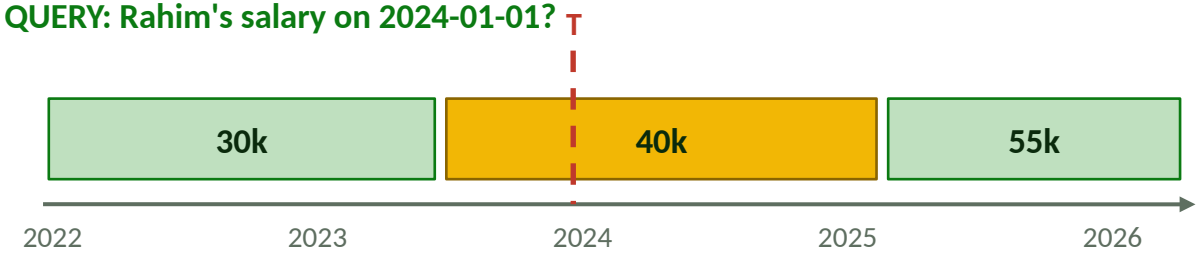
Valid time

when the fact was true in the real world (e.g. salary effective dates)

Transaction time

when the fact was stored in / removed from the database (audit trail)

QUERY: Rahim's salary on 2024-01-01?



The index jumps straight to the interval that contains T → 40,000, without scanning the full history.

EXAMPLE: EMPLOYEE SALARY HISTORY

Name	Salary	Valid from	Valid to
Rahim	30,000	2022-01-01	2023-06-30
Rahim	**40,000**	2023-07-01	2025-02-28
Rahim	55,000	2025-03-01	now

HOW IT IS INDEXED

- **B+ tree on (key, start time)** — simple and widely used
- **Interval tree** — finds all periods that contain T
- **Multi-version B-tree** — keeps old and new versions of each node

Used in: SQL:2011 system-versioned tables (SQL Server, MariaDB), bank audit logs, IoT / sensor time-series (TimescaleDB)

THE INDUSTRY-STANDARD ORDERED INDEX

Why Do We Need a B+ Tree?

THE PROBLEM WITH A PLAIN SORTED INDEX FILE

- Inserting a new key means shifting many entries to keep the order
- Overflow blocks pile up, so searches get slower over time
- The whole file must be reorganised now and then

WHAT THE B+ TREE DOES

- **Stays balanced by itself** — every search takes the same number of steps
- **Changes are local** — an insert/delete touches only a few nodes
- **Very shallow** — each node has many children (high fan-out)
- **Leaves are linked** — range queries just walk along the leaves

Think of it as a dictionary's guide words: the top tells you which section, the next level which page, and the page holds the word.

FIND 1 ROLL AMONG 1,000,000 STUDENTS

≈ 20

disk reads with binary search ($\log_2 10^6$)

≈ 4

disk reads with a B+ tree of order 100

HOW?

Each node has at least $\lceil 100/2 \rceil = 50$ children.

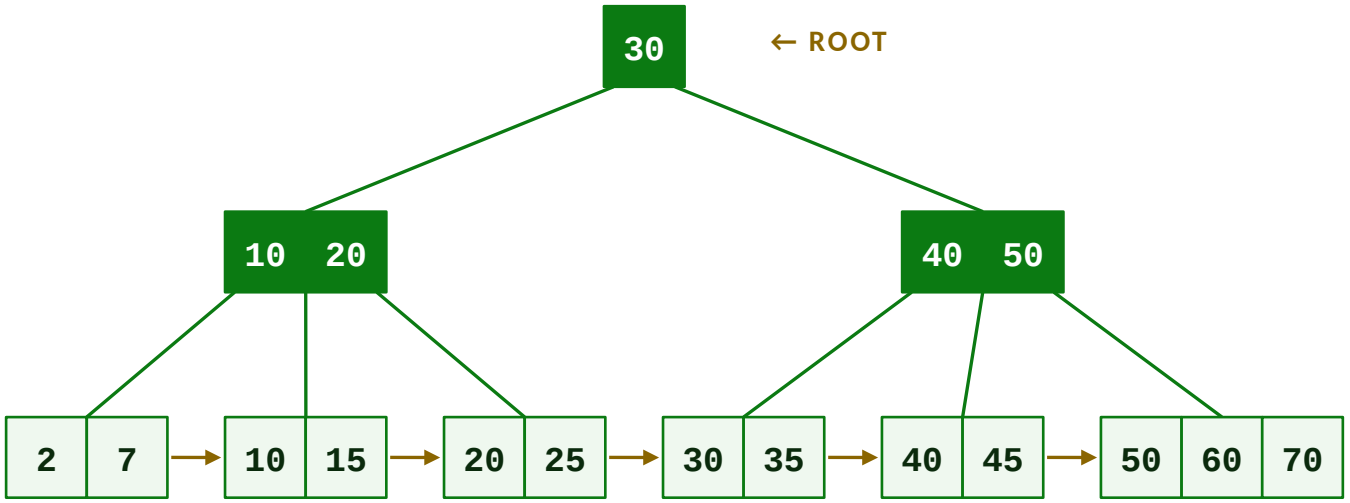
$$\text{Height} \le \lceil \log_{50}(1{,}000{,}000) \rceil = 4$$

since $50^4 = 6{,}250{,}000 \ge 1{,}000{,}000$.

One node = one disk block, so ~4 reads.

PARTS OF A B+ TREE

B+ Tree Structure



Reading a node: in internal node [10 | 20], the left pointer leads to keys < 10, the middle to 10 ≤ key < 20, the right to keys ≥ 20.

Every key lives in a leaf. Keys in internal nodes are copies used only for routing.

RULES FOR A B+ TREE OF ORDER n

Rule	Order n	n = 4
Max children per node	n	**4**
Max keys per node	n − 1	**3**
Min keys in a leaf	⌈(n−1)/2⌉	**2**
Min children, internal node	⌈n/2⌉	**2**
Min children of root	2	**2**
All leaves on same level	always	✓

All problems in this lecture use n = 4: a node holds at most 3 keys; a leaf must keep at least 2 (root excepted).

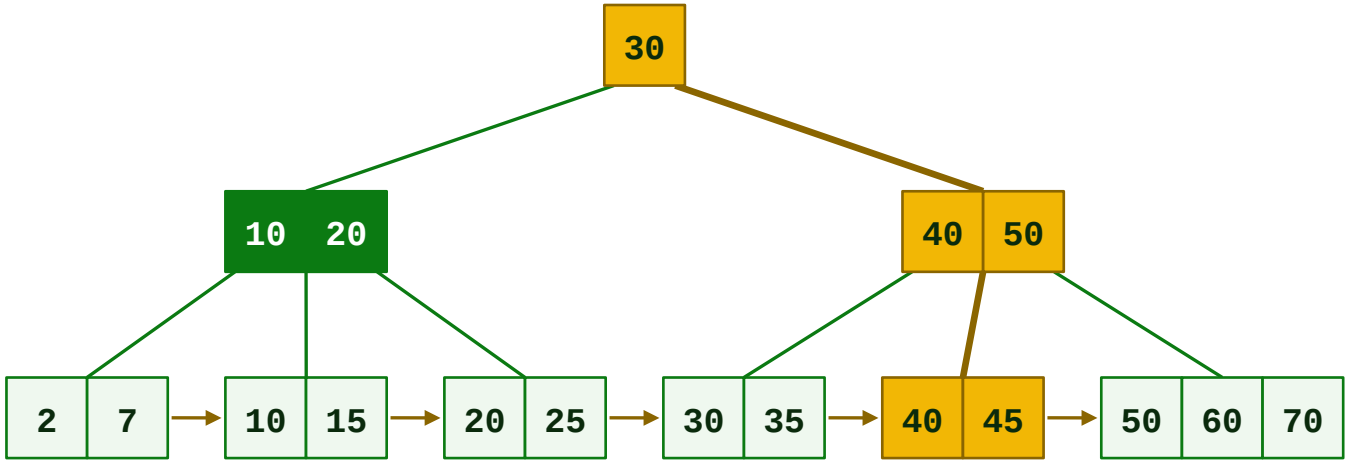
FOLLOW THE SIGNPOSTS DOWN

Searching a B+ Tree

POINT QUERY: find 45

1. **Root [30]**: 45 ≥ 30 → go right
2. **Node [40 | 50]**: 40 ≤ 45 < 50 → middle child
3. **Leaf [40 | 45]**: 45 found → follow its pointer to the row

Cost = height of tree = 3 node reads



RANGE QUERY: all keys between 15 and 42

1) Search for 15 as above → leaf [10 | 15]. **2)** Walk right along the leaf links, collecting keys until one is > 42.

15 | 20 | 25 | 30 | 35 | 40

stop at 45 (> 42). No need to go back up the tree.

Rule at each internal node with keys $K_1 \dots K_m$: key < K_1 → leftmost child; $K_i \le$ key < K_{i+1} → child between them; key ≥ K_m → rightmost child.

ADDING A KEY

B+ Tree Insertion — The Rules

1. **Find the leaf**

   Search for the key as usual to reach the correct leaf.

2. **Leaf has room?**

   Insert the key in sorted order. Done.

3. **Leaf is full → split**

   Put n keys in sorted order; left leaf keeps the first ⌈n/2⌉, right leaf gets the rest. COPY the first key of the right leaf up to the parent.

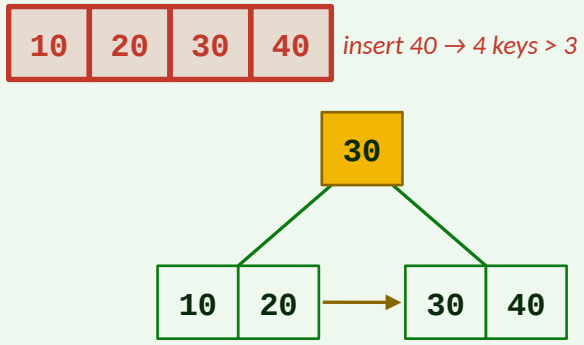
4. **Parent is full → split**

   Split the internal node; the middle key MOVES up (it does not stay in either half).

5. **Root splits**

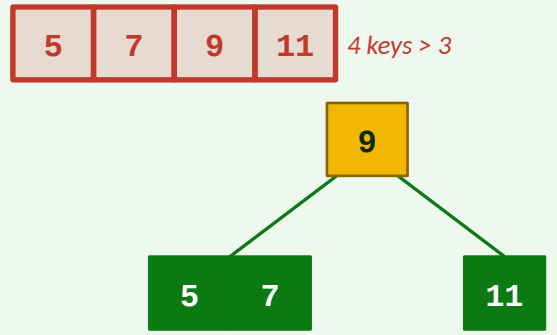
   A new root is created. The tree grows one level taller, at the top.

LEAF SPLIT — key is COPIED up



30 is in the parent AND still in the leaf

INTERNAL SPLIT — middle key MOVES up



9 goes up and is no longer in either half

Problem 1: Build a B+ Tree

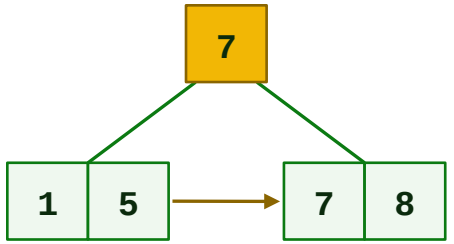
Problem 1 — Construct a B+ tree of order n = 4 (max 3 keys per node) by inserting: 8, 5, 1, 7, 3, 12, 9, 6, 10, 11

1 Insert 8, 5, 1



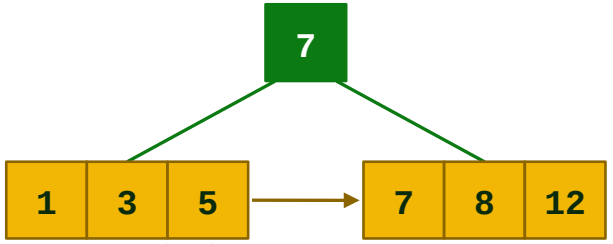
The root is a leaf. 3 keys fit, so just keep them sorted.

2 Insert 7 → leaf overflow, split



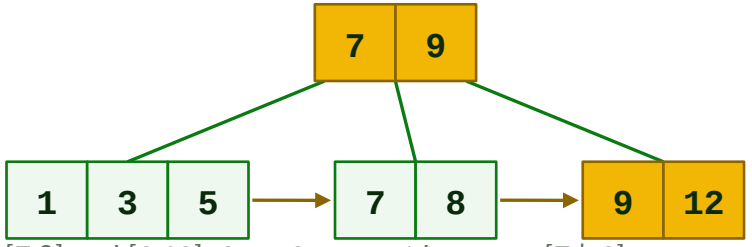
[1 5 7 8] splits into [1 5] and [7 8]. Copy 7 up as the new root.

3 Insert 3, then 12



3 goes left (< 7), 12 goes right (≥ 7). Both leaves have room.

4 Insert 9 → right leaf overflows

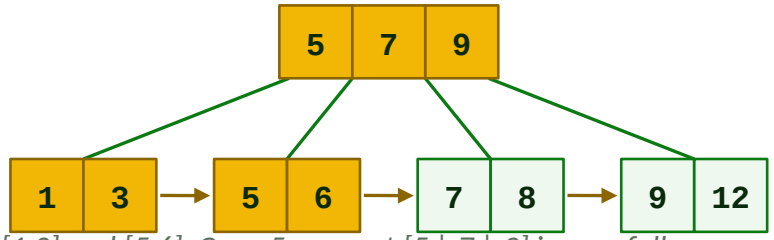


[7 8 9 12] → [7 8] and [9 12]. Copy 9 up; root becomes [7 | 9].

Problem 1: Build a B+ Tree (cont.)

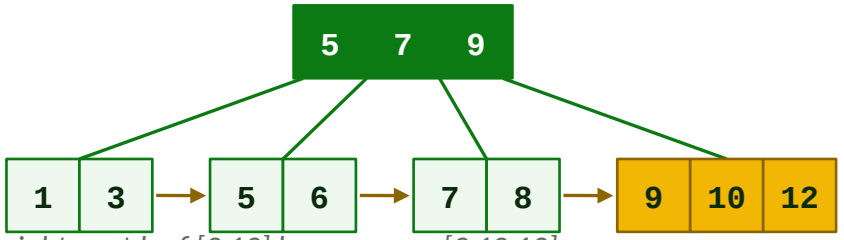
Problem 1 — Construct a B+ tree of order n = 4 (max 3 keys per node) by inserting: 8, 5, 1, 7, 3, 12, 9, 6, 10, 11

5 Insert 6 → left leaf overflows



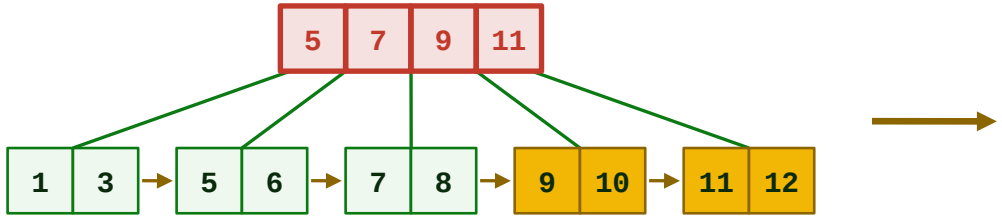
[1 3 5 6] → [1 3] and [5 6]. Copy 5 up; root [5 | 7 | 9] is now full.

6 Insert 10

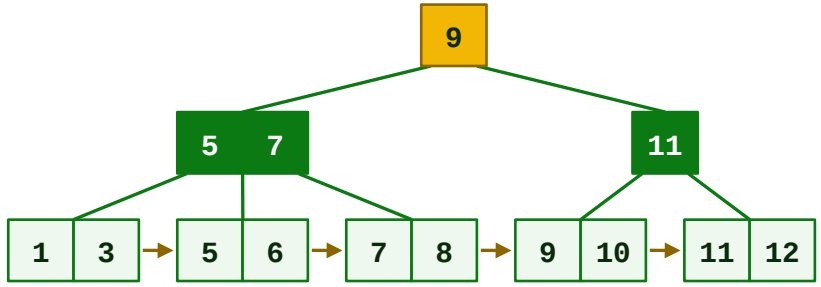


10 ≥ 9 → rightmost leaf [9 12] has room → [9 10 12].

7 Insert 11 → leaf split, then root split (tree grows taller)



a) [9 10 11 12] splits, 11 copied up → root has 4 keys (overflow!)



b) Root splits: [5 7] | 9 moves up | [11]. Final tree, height 3.

REMOVING A KEY

B+ Tree Deletion — The Rules

Find the leaf, remove the key, then check: does the leaf still have the minimum ($\lceil (n-1)/2 \rceil = 2$ keys for $n = 4$)?

1 Case 1 · No underflow

Leaf still has ≥ minimum keys. Just delete. Done.

2 Case 2 · Borrow (redistribute)

Leaf underflows, but a neighbour sibling has a spare key. Move one key across, then update the separator key in the parent.

3 Case 3 · Merge (coalesce)

Leaf underflows and the sibling has only the minimum. Merge the two leaves, then delete the separator key from the parent.

4 Case 4 · Merge travels up

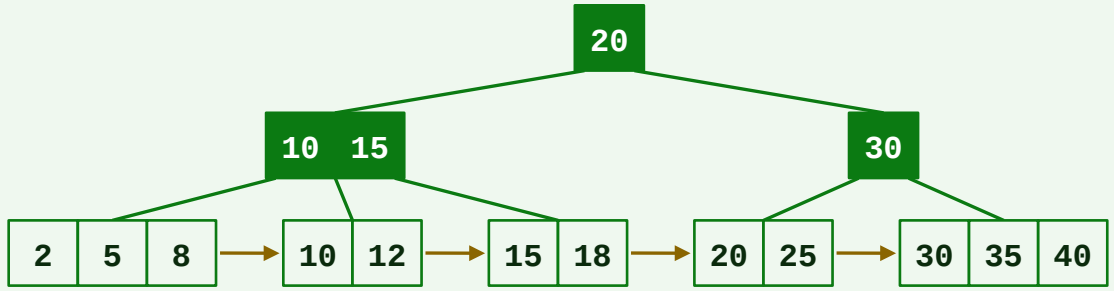
If the parent now underflows, repeat borrow/merge at that level (pulling the parent's separator key down). If the root is left with one child, remove it: the tree gets shorter.

Order to try: Case 1 → Case 2 (borrow) → Case 3 (merge). Borrow first, because merging changes more of the tree.

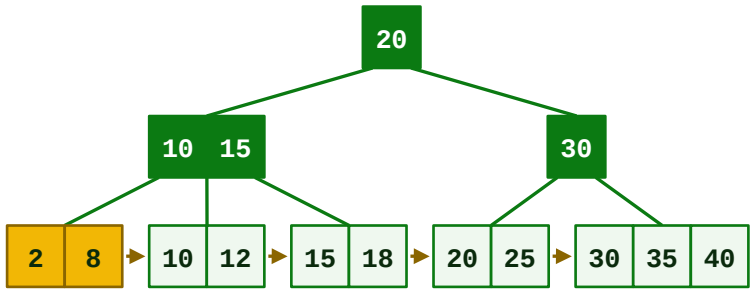
Problem 2: Delete from a B+ Tree

Problem 2 — Given the B+ tree (n = 4, min 2 keys per leaf), delete in order: 5, 25, 12, 35

GIVEN TREE

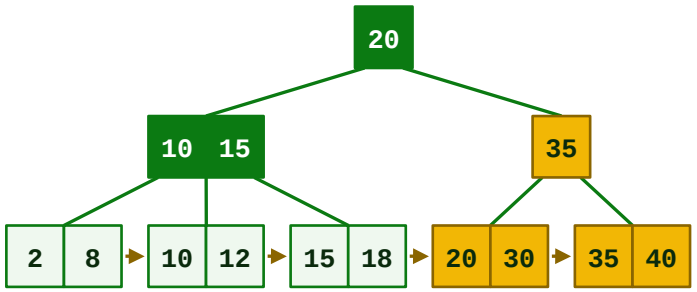


1 Delete 5 → Case 1 (no underflow)



Leaf [2 5 8] → [2 8] still has 2 keys. Nothing else changes.

2 Delete 25 → Case 2 (borrow)

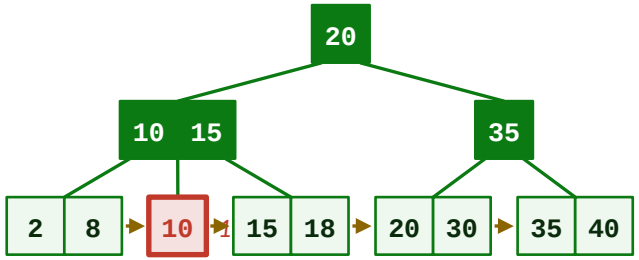


[20] underflows; right sibling [30 35 40] lends 30 → [20 30], [35 40]. Parent key 30 → 35.

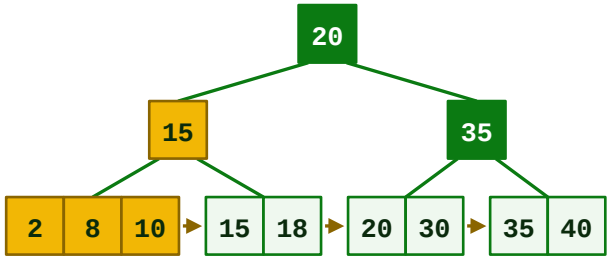
Problem 2: Delete from a B+ Tree (cont.)

Problem 2 — Given the B+ tree (n = 4, min 2 keys per leaf), delete in order: 5, 25, 12, 35

3 Delete 12 → Case 3 (merge leaves)

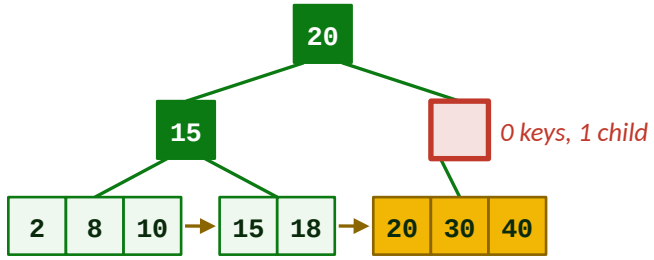


[10] underflows. Siblings [2 8] and [15 18] have only 2 keys, so nothing to borrow → merge [10] into [2 8]. Remove separator 10 from the parent.

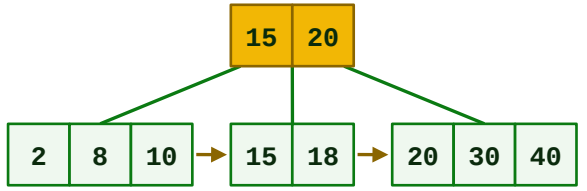


Parent [15] still has 2 children → OK, stop.

4 Delete 35 → Case 4 (merge goes up, height shrinks)



[40] underflows; sibling [20 30] is minimal → merge into [20 30 40]. Parent loses key 35 and underflows; its sibling [15] cannot lend → merge [15] + 20 (pulled down from root) + [ ] = [15 | 20]. Root is empty → removed.



Final tree: height 3 → 2.

TRY IT YOURSELF

Practice Problems

Q1 Draw

Construct a B+ tree of order n = 4 by inserting: 2, 31, 3, 29, 5, 23, 7, 19, 11, 17, 13. Show the tree after every split.

Q2 Delete

From your answer to Q1, delete 23, then 19, then 31. Name the case (no underflow / borrow / merge) for each.

Q3 Order 3

Insert 10, 20, 30, 40, 50, 60, 70, 80 into a B+ tree with n = 3 (max 2 keys). What is the final height?

Q4 Search

On your tree from Q1, list the nodes visited for the range query $5 \le \text{key} \le 19$.

Q5 Height

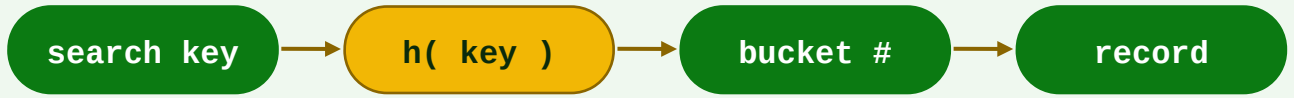
A B+ tree has order n = 200 and stores 4 million keys. At most how many levels does it have?

Q6 Concept

Which index would you use for: (a) “restaurants within 1 km”, (b) “account balance on 1 Jan 2025”? Why?

DIRECT ACCESS BY KEY — NO ORDERING

Static & Dynamic Hashing



A hash function sends a key straight to a bucket: about 1 disk read on average, but no range queries (keys are not kept in order).

Static Hashing

fixed number of buckets

- **Simple:** bucket count B is chosen up front
- **Overflow chains:** when a bucket is full, extra records go to linked overflow buckets
- **Gets slow:** as the file grows, chains get long
- **Fix = rehash** the whole file into more buckets

Dynamic Hashing

buckets grow & shrink on demand

- **Extendable hashing:** a directory uses the first i bits of h(key)
- **Split, don't chain:** a full bucket splits; the directory doubles only when needed
- **Scales smoothly** as the data grows or shrinks
- **Linear hashing:** adds buckets one at a time, no directory

Rule of thumb: exact-match lookups (WHERE id = 7) → hashing; ranges, sorting, BETWEEN, ORDER BY → B+ tree.

WRAP-UP

Key Takeaways

Index

like a book index: search key + pointer; trades a little space for far fewer disk reads.

Dense / sparse, primary / secondary

dense = every key, sparse = one per block; primary follows file order.

Spatial index

R-tree, quad-tree or grid for “what is near here?” queries on maps.

Temporal index

finds the version of a row valid at time T (valid / transaction time).

B+ tree

balanced, shallow, all keys in linked leaves: fast point and range search.

Insert / delete

insert splits (leaf copies up, internal moves up); delete borrows or merges.